

Lista de Exercícios | Árvores Binárias | Estrutura de Dados II

Monitora: Arianny

9 de setembro de 2026

Esta lista dá continuidade aos Exercícios Práticos 1–6 do notebook da Semana 03. Seguindo a mesma convenção do professor, as funções abaixo são funções soltas que recebem o nó (ou os nós) como parâmetro, exatamente como em `buscar(no, valor)` ou `sao_identicas(no1, no2)`. Presume-se apenas a classe `No` (com `valor`, `esquerda` e `direita`).

Tente resolver cada questão sozinho(a) antes de consultar a resolução, disponível em [resolucoes/](#) neste mesmo repositório. As questões possuem níveis de dificuldade diferentes e algumas foram pensadas como aprofundamento. As dicas são opcionais; use-as apenas se necessário.

As questões estão divididas dessa maneira:

Questão	Nível
1 – Soma dos Níveis Pares	Fácil
2 – Todos os Caminhos com Soma Alvo	Médio
3 – Árvore Espelhada sem Modificar a Original	Médio
4 – Menor Ancestral Comum	Médio/Difícil
5 – Árvore Balanceada	Médio
6 – Diâmetro da Árvore	Difícil
7 – Achatar a Árvore (Pré-ordem)	Médio/Difícil
8 – Serialização e Desserialização	Difícil

Questão 1 Soma dos Níveis Pares

(Fácil)

Conteúdo Relacionado: Percurso em Largura (BFS).

Enunciado: Implemente a função `soma_niveis_pares(raiz)` que retorna a soma de todos os valores localizados em níveis pares da árvore, considerando a raiz como nível 0 (portanto, nível par). Caso a árvore esteja vazia, retorne 0.

Dica: Em `largura(raiz)`, a fila já determina a ordem em que os nós são visitados. Agora imagine que, além do nó, cada elemento da fila carregasse uma segunda informação: quantas arestas foi preciso percorrer desde a raiz para chegar até ele. O que essa informação representa?

Questão 2 Todos os Caminhos com Soma Alvo

(Médio)

Conteúdo Relacionado: DFS e Backtracking.

Enunciado: Implemente a função `caminhos_com_soma(no, alvo)` que retorna uma lista de listas, onde cada lista interna contém os valores de um caminho da raiz até uma folha cuja soma seja exatamente igual a `alvo`. Pode existir mais de um caminho válido.

Dica: Observe novamente `caminho_ate_valor`: o `append` registra a entrada em um nó e o `pop` desfaz essa escolha ao retornar da recursão. Aqui, encontrar um caminho válido não deve encerrar a busca. Como guardar uma cópia do caminho encontrado e continuar explorando as outras possibilidades?

Questão 3 Árvore Espelhada sem Modificar a Original

(Médio)

Conteúdo Relacionado: Recursão com Construção de Nova Estrutura.

Enunciado: Implemente a função `arvore_espelhada(no)` que retorna a raiz de uma **nova** árvore, espelho da original, sem modificar em nada os nós recebidos como entrada.

Dica: Em `espelhar_arvore(no)`, os próprios nós recebidos são modificados. Mas e se a regra for não tocar em nenhum deles? Pense no caso mais simples: ao visitar um nó de valor `x`, como criar outro nó com o mesmo valor e fazer com que a subárvore que estava à direita passe a ocupar o lado esquerdo da nova árvore?

Questão 4 Menor Ancestral Comum

(Médio/Difícil)

Conteúdo Relacionado: Recursão com Retorno de Referência de Nó.

Enunciado: Implemente a função `menor_ancestral_comum(no, valor1, valor2)` que retorna o valor do menor ancestral comum entre dois valores existentes na árvore.

Dica: Pense no que uma chamada recursiva poderia devolver ao terminar de examinar uma subárvore. Se a busca pela esquerda encontra um dos valores e a busca pela direita encontra o outro, há algo especial no nó atual. Agora pense no caso em que apenas um dos lados devolve alguma coisa: para onde essa informação deve continuar subindo?

Questão 5 Árvore Balanceada

(Médio)

Conteúdo Relacionado: Recursão com Retorno Múltiplo.

Enunciado: Implemente a função `eh_balanceada(no)` que retorna `True` se, para **todo** nó da árvore, a diferença de altura entre as subárvores esquerda e direita for no máximo 1.

Dica: A função de altura já precisa visitar as subárvores antes de calcular a altura do nó atual. Nesse mesmo momento, você já conhece as alturas dos dois lados. Será mesmo necessário percorrer essas subárvores novamente para descobrir se elas estão balanceadas? Pense em fazer cada chamada recursiva devolver, de uma só vez, tanto a informação de balanceamento quanto a altura calculada.

Questão 6 Diâmetro da Árvore

(Difícil)

Conteúdo Relacionado: Acumulador Externo (`nonlocal`) e Recursão de Altura.

Enunciado: Implemente a função `diametro(no)` que retorna o número de arestas do maior caminho entre quaisquer dois nós da árvore. Esse caminho pode ou não passar pela raiz.

Dica: Para cada nó, tente responder duas perguntas diferentes: “qual é a altura que devo informar ao meu pai?” e “qual é o maior caminho que passa por mim?”. A primeira informação precisa subir pela recursão; a segunda precisa ser comparada com o melhor resultado encontrado até aquele momento.

Questão 7 Achatar a Árvore (Pré-ordem)

(Médio/Difícil)

Conteúdo Relacionado: Percurso Pré-ordem e Modificação In-Place.

Enunciado: Implemente a função `achatar(no)` que reorganiza a árvore, no próprio lugar (*in-place*), transformando-a em uma “lista encadeada” que segue a ordem de um percurso pré-ordem: cada nó deve apontar para o próximo apenas através de `direita`, com `esquerda` igual a `None` em todos os nós ao final.

Dica: Antes de alterar qualquer ponteiro, pense no que poderia ser perdido. Depois que a subárvore esquerda for achatada, ela deve aparecer imediatamente após o nó atual na sequência de pré-ordem. Mas onde a antiga subárvore direita deverá ser conectada? Talvez seja útil saber onde termina a parte que acabou de ser achatada.

Questão 8 Serialização e Desserialização

(Difícil)

Conteúdo Relacionado: Percurso Pré-ordem e Reconstrução de Estruturas.

Enunciado: Implemente a função `serializar(no)`, que converte a árvore em uma string, e a função `desserializar(texto)`, que reconstrói a raiz de uma árvore a partir dessa string. A reconstrução deve produzir exatamente a mesma estrutura da árvore original.

Dica: Escrever apenas os valores visitados não é suficiente para reconstruir a forma da árvore. Por exemplo, árvores diferentes podem produzir sequências semelhantes se os filhos ausentes forem ignorados. Pense em duas coisas: qual percurso visita o nó antes de suas subárvores e qual **marcador** poderia registrar explicitamente que uma determinada posição não possui filho.
